The Manchester Thermal Analyzer

Version 2.0

Yi-Chung Chen* Scott Ladenheim*† Milan Mihajlović*‡ Vasilis Pavlidis*§

August 24, 2017

Contents

1 Introduction **2**

2 New features **3**

3 Download and installation **3**

4 Quick tutorial **4**
4.1 Example 1: Steady-state analysis of the ev6 circuit executed in parallel . . . 4
4.2 Example 2: Transient analysis involving spatiotemporal adaptivity . . . 9
4.3 Example 3: Nonlinear transient analysis . . . 10

5 Tool Overview **12**

6 Run flow script configuration **14**

7 XML Module **16**
7.1 Block-level description . . . 16
7.1.1 Supported geometries . . . 18
7.2 Cell-level description . . . 20

8 Thermal simulation module **22**
8.1 Mathematical details . . . 22
8.1.1 The Poisson equation . . . 22
8.1.2 The linear heat equation . . . 24
8.1.3 Adaptive spatiotemporal refinement . . . 25

*School of Computer Science, University of Manchester, Manchester, UK, M13 9PL (yi-chung.chen@manchester.ac.uk)
†(scott.ladenheim@manchester.ac.uk)
‡(milan.mihajlovic@manchester.ac.uk)
§(vasileios.pavlidis@manchester.ac.uk)

8.1.4 The nonlinear heat equation . . . 27
8.1.5 The linearized heat model . . . 28
8.2 Thermal configuration flags . . . 29
8.2.1 Mode 0 flags . . . 29
8.2.2 Mode 1 flags . . . 29
8.2.3 Mode 2 flags . . . 30
8.2.4 Mode 3 flags . . . 30
8.2.5 Mode 4 flags . . . 30
8.2.6 Model type flag . . . 31
8.2.7 Output flags . . . 31
8.2.8 Checkpoint flags . . . 31

9 Mesh generation module **31**
9.1 Mesh refinement flags . . . 32
9.2 Cell-level mesh reduction flags . . . 34
9.2.1 Spatially weighted grid lines . . . 34
9.2.2 Power density weighted grid lines . . . 34
9.2.3 Hybrid mode . . . 34

10 External input files **35**
10.1 Powertrace . . . 35
10.2 DEF and LEF . . . 35

1 Introduction

The Manchester Thermal Analyzer (MTA) version 2.0 is an open-source academic tool for performing steady-state and transient thermal analysis of two- and three-dimensional integrated circuits (ICs) and their packaging. The MTA is a powerful and versatile tool delivering fast and accurate thermal analysis at both the block (floorplan) and cell-level granularity of a circuit. The tool consists of several interdependent user-configurable modules that execute the thermal analysis.

The MTA is designed as a user-friendly system that effectively automates the thermal analysis. The physical and material properties of an IC and the associated package are described in an XML file. Thermal simulations are then performed by numerically solving the heat equation using the finite element method (FEM) and implicit time stepping methods. The MTA includes a fast mesh generator which parses the dimensions of the circuit and its component parts described in the XML file to create a finite element mesh. Power information is supplied to the MTA from external powertrace files.

The core of this manual describes the three modules of the MTA: XML, mesh generation, and thermal simulation. Interacting with these modules and configuring the thermal analysis is achieved through the Perl script `run_flow.pl`. In addition, pertinent user information is provided which includes descriptions of the various flags used to customize the tool and computational details of the underlying mathematical model.

The manual is organized as follows. In Section 2, we provide a list of the newly available features that have been added to version 2.0 of the MTA. In Section 3, the download and installation information for the MTA is given. In Section 4, we provide a quick tutorial of the tool where we work through a several example problems to demonstrate how to execute thermal simulations and enable various computational features. This allows the user to easily and quickly understand how to operate the MTA.

An overview of the tool is provided in Section 5. In Section 6, the run flow Perl script for executing thermal analyses in the MTA is described. This script controls the execution flow of the thermal analysis and is crucial for operating the MTA. The XML module is described in Section 7. This section describes how to create XML files representing circuits at both the block and cell-level. This section is important for users wishing to create and test their own designs as customized XML files.

In Section 8, the thermal simulation module is described. We present the computational routines used to perform thermal simulations in the MTA and provide details on how to enable features designed to improve their computational efficiency. In addition, mathematical details for the utilized numerical methods are reviewed. In Section 9, the mesh generation module is described. This section provides users with information on how to create quality meshes for their thermal simulations. Lastly, in Section 10, we describe the supported input files to the MTA.

Thank you for choosing our tool, we hope you enjoy using it. Any comments and feedback which could improve the capabilities and user's experience are greatly appreciated and can be directed to the email address mta-support@listserv.manchester.ac.uk. The acknowledgement for using the MTA is provided by citing the reference [21].

2 New features

Listed below are the new features included in the MTA version 2.0 compared to version 1.0 published in December, 2016.

- The capability to perform *nonlinear thermal simulations* where the material properties of the circuit are temperature dependent. Mathematical details are provided in Section 8.1.4 and how to set up nonlinear thermal simulations is discussed in Section 8.2.6.
- Improved adaptive time stepping routines. The MTA now offers the implicit midpoint rule so as to provide conservative schemes that accurately reproduce the system's energy in nonlinear thermal simulations (Section 8.1.2).
- Functionality to perform standard cell-level thermal analysis and new homogenization techniques aimed at reducing the size of the computational mesh to perform more computationally efficient thermal analyses (Section 7.2).
- Improved and more efficient features for outputting solution related data (Section 8.2.7).
- The checkpoint capability to stop and restart transient thermal simulations (Section 8.2.8).
- The lateral boundaries of the circuit can now be modeled either with Neumann or Robin boundary conditions (equation (8.11b)). Non-adiabatic lateral boundary conditions can be set in the XML file (Section 7).

3 Download and installation

The MTA is publicly available to download at

`https://staffnet.cs.manchester.ac.uk/acso/thermal-analyzer/`

Documentation for installing the tool and getting started is provided on the website. After downloading, unzip the file `mta.tar.gz`. This creates the directory `mta/` (in the current directory). Inside this directory is

a `README.txt` file, the Perl script `run_flow.pl`, and three subdirectories `bin/`, `license/`, and `examples/`. The `bin/` subdirectory contains the binary executables required by the tool, the `license/` subdirectory contains licensing information, and the `examples/` subdirectory contains XML and powertrace files for some example benchmark problems; refer to the Gallery tab on the website for further information on these XML and powertrace files.

4 Quick tutorial

The MTA can be easily used without any prior knowledge of the underlying code or associated mathematics. The execution of the program is accomplished with the Perl script `run_flow.pl` in the `mta/` directory. Thermal simulations are performed with the command:

perl run_flow.pl -xml <**.xml**> **-ptrace** <**.ptrace**> **-mode** <**m**> **-option_flag** <**flag**>. (4.1)

The `run_flow` Perl script requires an **-xml** flag followed by an XML file. For thermal simulations the **-ptrace** flag followed by the powertrace file is also required. The user sets the operation mode with the **-mode** flag followed by an integer $\mathbf{m} = 0, 1, 2, 3, 4$. If the user does not specify a mode, the MTA defaults to transient analysis in Mode 1 using a fixed spatial grid and fixed time step size.

Additional option flags to configure the thermal analysis flow can be passed to the Perl script at the command line. Descriptions of these flags are given in Section 6. For example, to execute thermal simulations in parallel, use the flag and argument pair **-mpi** <**N**>, where **N** is the number of processors. The default number of processors is $\mathbf{N} = 1$.

To familiarize users with the MTA, we present three tutorial problems to provide an overview of how to use important features in the tool. In this tutorial, we consider two benchmark problems, the ev6 benchmark [25] and a synthetic nonlinear benchmark that are both described in the XML gallery on the website. The first benchmark is used to illustrate the different operation modes of the tool using a linear model and the second benchmark shows how to execute nonlinear thermal simulations.

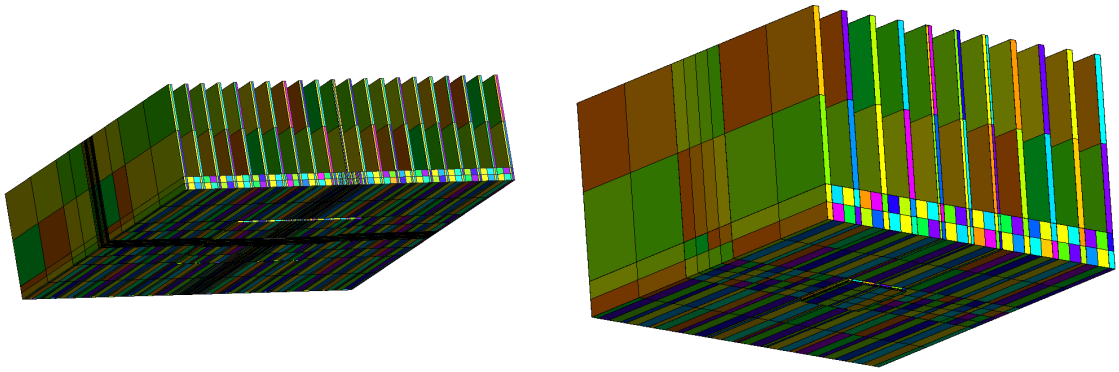
The files for the ev6 benchmark are located in the `examples/ev6/` subdirectory. The ev6 benchmark models an integrated circuit and package. The circuit consists of an active layer, substrate, and thermal interface material (TIM) layer that is attached to an overhanging heat spreader and heat sink with fins. The ev6.ptrace file contains ten rows of power values supplied to various regions of the active layer of the circuit every 3.33 microseconds (3.33×10^{-6} seconds). This base unit of time is called the *atomic time step size* of the simulation. For steady-state simulations the average power is used as the input power source.

The files for the synthetic nonlinear example are located in the `examples/syn_nl` subdirectory. This benchmark problem models an integrated circuit consisting of two active silicon tiers, subdivided into 4 equal sized cores, and attached to a heat spreader and finned heat sink. The powertrace file randomly applies either 25W or 10W of power to different cores of the circuit for 10 time steps with an atomic time step size of 10 milliseconds for a total simulation time of 0.1 seconds. The finite element meshes generated for the benchmark problems are displayed in Figure 1. Further details on the XML and powertrace files used in both of these benchmarks is provided in the Gallery on the MTA website.

4.1 Example 1: Steady-state analysis of the ev6 circuit executed in parallel

A steady-state simulation of the ev6 benchmark problem is executed with the command

perl run_flow.pl -xml examples/ev6/ev6.xml -ptrace examples/ev6/ev6.ptrace -mode 0.



(a) ev6 benchmark (b) Synthetic benchmark for nonlinear simulation

Figure 1: FEM meshes for the benchmark problems.

The displayed information on the screen is shown in Figure 2. The screen output reflects the flow of the MTA, namely, the XML and powertrace files are parsed first, then the mesh is generated, and finally the thermal simulation is performed.

The number of elements (i.e., hexahedrons/cuboids) and the degrees of freedom (DOF), which is the total number of nodes in the mesh, are displayed at the start of the thermal simulation. The simulation data displays the type of heat flow model and the order of approximation in the finite element. For this example a linear thermal simulation (i.e., with temperature-independent material properties) with Q1, or first order elements has been executed. The order of the element used determines the spatial accuracy of the numerical scheme.

A summary of the times spent in each phase of the program is also displayed. This contains information on the assembly and solve times, the time to output the solution, and the time for various other setup costs. The **solution.pvd** file in the newly created `Output` subdirectory is the steady-state temperature profile and can be visualized with the open-source software Paraview [1]. Several viewpoints of the solution are displayed in Figure 3. For further information on Paraview refer to the online documentation.

Thermal simulations are executed in parallel by setting the **-mpi** flag followed by the number of processors **N** at the command line. For example, the command

perl run_flow.pl -xml ev6.xml -ptrace ev6.ptrace -mode 0 -mpi 2,

executes the previous simulation in parallel with $\mathbf{N} = 2$ processors. The full subdirectory names are omitted for notational brevity. The terminal output for this parallel simulation is shown in Figure 4. Note that in the output the number of processors is now set to 2 and the overall simulation time decreases.

To configure the execution (flow) of the thermal analysis the user can add option flags to the Perl script at the command line and a full list of these flags is given in Section 6. To configure the (steady-state) thermal simulation in the MTA, a user adds flags to the **$heat_configuration$** variable in the `run_flow` Perl script. How to modify this variable is described in Section 6 and the full list of option flags for Mode 0 simulations is given in Section 8.2.1.

```
READIN/OUTPUT XML: examples/ev6/ev6.xml
READIN PTRACE: examples/ev6/ev6.ptrace
SET MTA in MODE 0
WARNING: undefined .msh file name
Using default mesh file name defaultMeshName.msh.
Mesh Generation: bin/msh_gen -xml examples/ev6/ev6.xml -mesh Output/
defaultMeshName.msh
Thermal Simulation:  bin/heat -xml examples/ev6/ev6.xml -mesh Output/
defaultMeshName.msh -ptrace examples/ev6/ev6.ptrace  -steady -folder Output

============================================================
Number of elements: 64288
Number of degrees of freedom: 78035

Simulation data:
     Model type linear
     Element Q1


+-----------------------------------------+------------+------------+
| Total wallclock time elapsed since start |      5.81s |            |
|                                         |            |            |
| Section                          | no. calls |  wall time | % of total |
+----------------------------------+-----------+------------+------------+
| Assemble Poisson system          |         1 |     0.544s |       9.4% |
| Distribute dofs                  |         1 |     0.221s |       3.8% |
| Make grid                        |         1 |     0.926s |        16% |
| Output solution                  |         1 |      3.65s |        63% |
| Set environment                  |         1 |   0.00214s |     0.037% |
| Solve system                     |         1 |     0.469s |       8.1% |
+----------------------------------+-----------+------------+------------+
```

Figure 2: Mode 0 output.

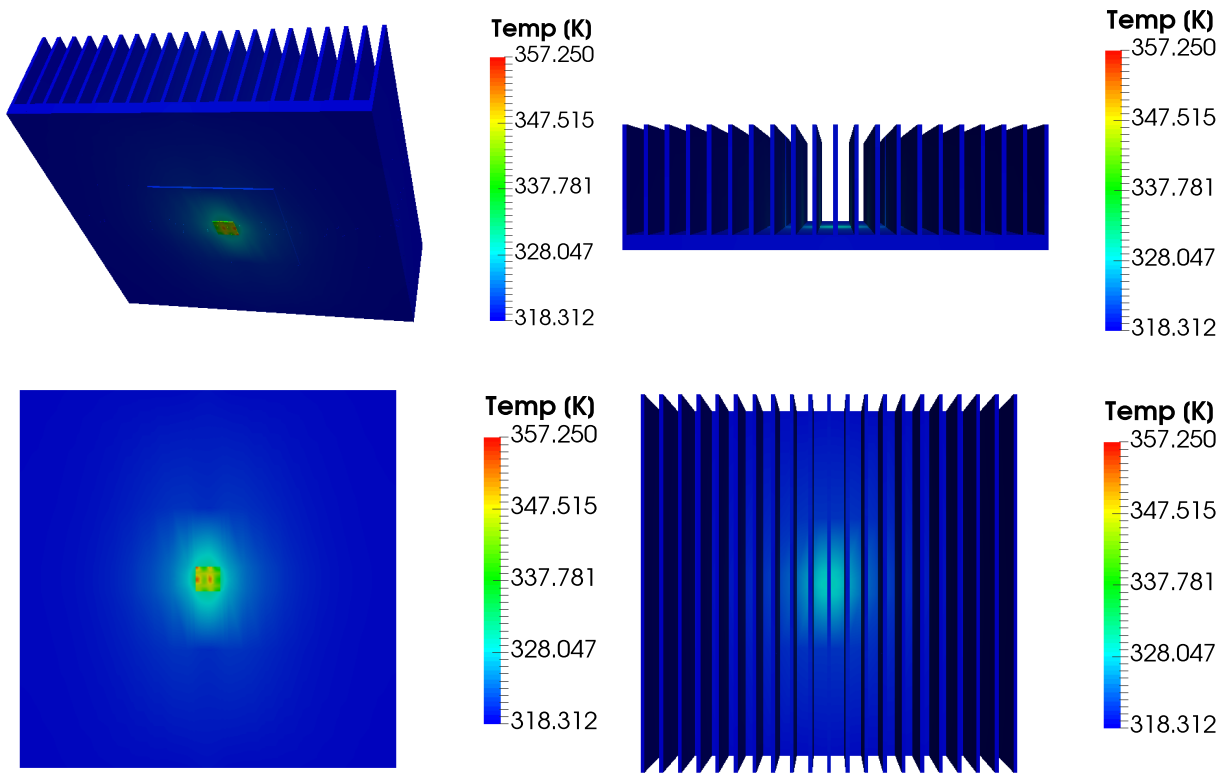


Figure 3: Steady-state temperature distribution for the ev6 benchmark circuit.

```
READIN/OUTPUT XML: ev6/ev6.xml
READIN PTRACE: ev6/ev6.ptrace
SET MTA in MODE 0
RUN MPI in NP = 2
WARNING: undefine of .msh file
Using default mesh name - defaultMeshName.msh.
Mesh Generation: ./msh_gen -xml ev6/ev6.xml -mesh Output/defaultMeshName.msh
Thermal Simulation: mpirun -np 2 ./heat -xml ev6/ev6.xml -mesh Output/defaultMeshName.msh
-ptrace ev6/ev6.ptrace -verbose -model_type linear -timestep_method be -steady -folder Out
put

============================================================
Number of elements: 64288
Number of degrees of freedom: 78035

Simulation data:
     Model type linear
     Element Q1
     System matrix solved in 18 CG iterations.


+---------------------------------------------+------------+------------+
| Total wallclock time elapsed since start    |      3.53s |            |
|                                             |            |            |
| Section                          | no. calls |  wall time | % of total |
+----------------------------------+-----------+------------+------------+
| Assemble Poisson system          |         1 |     0.294s |       8.3% |
| Distribute dofs                  |         1 |     0.144s |       4.1% |
| Make grid                        |         1 |      1.06s |        30% |
| Output solution                  |         1 |      1.72s |        49% |
| Set environment                  |         1 |   0.00155s |         0% |
| Solve system                     |         1 |     0.318s |         9% |
+----------------------------------+-----------+------------+------------+
```

Figure 4: Output of Mode 0 executed on two processors.

4.2 Example 2: Transient analysis involving spatiotemporal adaptivity

The command

perl run_flow.pl -xml ev6.xml -ptrace ev6.ptrace -mode 1

executes a transient thermal simulation in Mode 1. Mode 1 simulations solve the heat equation with fixed spatial and temporal discretizations. The information displayed to the screen for this transient simulation is shown in Figure 5. Note that additional information compared to the steady-state case is output to the screen. This information gives the final time of the simulation, the utilized time step method, and whether the adaptive spatiotemporal refinement routines have been enabled. For Mode 1 simulations where the spatial grid and time step size are fixed, these features are disabled (indicated by the number 0). For transient simulations the time step number, time, and corresponding time step size are also displayed.

```
READIN/OUTPUT XML: examples/ev6/ev6.xml
READIN PTRACE: examples/ev6/ev6.ptrace
SET MTA in MODE 1
WARNING: undefined .msh file name
Using default mesh file name defaultMeshName.msh.
Mesh Generation: bin/msh_gen -xml examples/ev6/ev6.xml -mesh Output/
defaultMeshName.msh
Thermal Simulation:  bin/heat -xml examples/ev6/ev6.xml -mesh Output/
defaultMeshName.msh -ptrace examples/ev6/ev6.ptrace   -folder Output

=========================================================
Number of elements: 64288
Number of degrees of freedom: 78035

Simulation data:
     Model type linear
     Tmax 2.9997e-05
     dt 3.333e-06
     Timestep method Backward Euler
     Element Q1
     Adaptive mesh refinement 0
     Adaptive time integration 0

Time step 1 at t=3.333e-06 with timestep dt 3.333e-06
Time step 2 at t=6.666e-06 with timestep dt 3.333e-06
Time step 3 at t=9.999e-06 with timestep dt 3.333e-06
Time step 4 at t=1.3332e-05 with timestep dt 3.333e-06
Time step 5 at t=1.6665e-05 with timestep dt 3.333e-06
Time step 6 at t=1.9998e-05 with timestep dt 3.333e-06
Time step 7 at t=2.3331e-05 with timestep dt 3.333e-06
Time step 8 at t=2.6664e-05 with timestep dt 3.333e-06
Time step 9 at t=2.9997e-05 with timestep dt 3.333e-06


+-------------------------------------------+------------+------------+
| Total wallclock time elapsed since start  |      11.5s |            |
|                                           |            |            |
| Section                          | no. calls |  wall time | % of total |
+----------------------------------+-----------+------------+------------+
| Assemble right hand side         |         9 |      1.09s |       9.5% |
| Assemble system matrix           |         1 |     0.973s |       8.5% |
| Distribute dofs                  |         1 |     0.221s |       1.9% |
| Make grid                        |         1 |      0.95s |       8.3% |
| Output solution                  |         2 |      5.21s |        45% |
| Project temperature field        |         1 |   0.00138s |     0.012% |
| Set environment                  |         1 |   0.00218s |     0.019% |
| Solve system                     |         9 |      2.42s |        21% |
+----------------------------------+-----------+------------+------------+
```

Figure 5: Mode 1 output.

The user can enable the adaptive spatiotemporal refinement features within the MTA by simply changing the mode. Mode 2 simulations use fixed spatial grids but the time step size is adaptively estimated. Mode 3 simulations have fixed time step sizes but adaptively refine the finite element mesh periodically. Lastly, Mode 4 simulations enable both of these features simultaneously. The mathematical details of the spatiotemporal refinement features are presented in the appropriately labeled subsections of Section 8.1.

Similarly to steady state simulations, transient thermal simulations can be further configured by adding flags to the **$heat_configuration$** variable in the `run_flow` Perl script. This is where users can set, for example, mode specific parameters and how often to output additional simulation data. These additional option flags are described in the subsections of Section 8.2.

4.3 Example 3: Nonlinear transient analysis

The operating temperature range of modern IC technologies is typically between 300-400 K [11]. The thermal conductivity and specific heat of the silicon components of a circuit are temperature-dependent and vary over this temperature range and therefore a linear model with temperature-independent material properties may not accurately represent the underlying physics of the problem and be able to capture the developing hotspots in a circuit. For this reason the MTA supports nonlinear thermal simulations.

Here, we demonstrate how to set up nonlinear thermal simulations for a Mode 1 simulation with the previously described synthetic benchmark problem. This is accomplished by setting the flag **-mode_type** in the **$heat_configuration$** to the value nonlinear, i.e., setting

$heat_configuration$ = **-model_type nonlinear**.

The command to execute the nonlinear thermal simulation is the same as before

perl run_flow.pl -xml syn_nl.xml -ptrace syn_nl.ptrace -mode 1.

The thermal profile at the end of this nonlinear simulation (T=0.1 sec) is displayed in Figure 6.

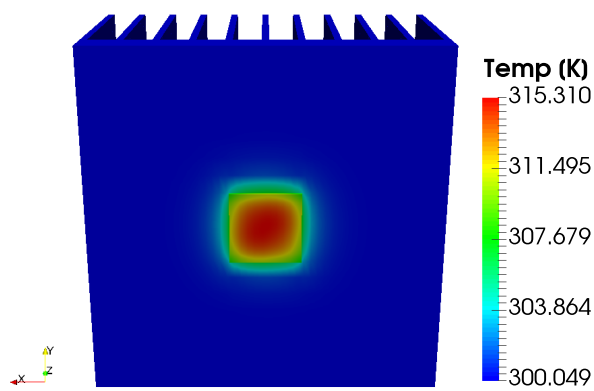


Figure 6: Temperature map for Example 4.3.

The output to the screen during the simulation is shown in Figure 7. The key differences with the previous output are that the mesh generator refines the grid prior to the simulation and that the model type is now nonlinear. The additional refinements are because the initial mesh is too coarse and to avoid inaccuracies close to the corners of the domain associated with the heat spreader. Due to the larger size of this problem and the repeated assembly of the Jacobian matrix and residual vector, the total simulation time is longer. Nonlinear problems are inherently more expensive to compute than linear problems and result in longer simulation times.

```
READIN/OUTPUT XML: examples/syn_nl/syn_nl.xml
READIN PTRACE: examples/syn_nl/syn_nl.ptrace
SET MTA in MODE 1
WARNING: undefined .msh file name
Using default mesh file name defaultMeshName.msh.
Mesh Generation: bin/msh_gen -xml examples/syn_nl/syn_nl.xml -mesh Output/defaultMeshName.msh
Need a automatic MESH refinement (DOF > 50000). Current DOF is 10598
Refine Mesh Generation: bin/msh_gen -xml examples/syn_nl/syn_nl.xml -mesh Output/defaultMeshName.msh  -refine 1
Need a automatic MESH refinement (DOF > 50000). Current DOF is 32237
Refine Mesh Generation: bin/msh_gen -xml examples/syn_nl/syn_nl.xml -mesh Output/defaultMeshName.msh  -refine 2
Thermal Simulation:  bin/heat -xml examples/syn_nl/syn_nl.xml -mesh Output/defaultMeshName.msh -ptrace examples/
syn_nl/syn_nl.ptrace -model_type nonlinear  -folder Output

==========================================================
Number of elements: 101032
Number of degrees of freedom: 114907

Simulation data:
     Model type nonlinear
     Tmax 0.1
     dt 0.01
     Timestep method Backward Euler
     Element Q1
     Adaptive mesh refinement 0
     Adaptive time integration 0

Time step 1 at t=0.01 with timestep dt 0.01
Time step 2 at t=0.02 with timestep dt 0.01
Time step 3 at t=0.03 with timestep dt 0.01
Time step 4 at t=0.04 with timestep dt 0.01
Time step 5 at t=0.05 with timestep dt 0.01
Time step 6 at t=0.06 with timestep dt 0.01
Time step 7 at t=0.07 with timestep dt 0.01
Time step 8 at t=0.08 with timestep dt 0.01
Time step 9 at t=0.09 with timestep dt 0.01
Time step 10 at t=0.1 with timestep dt 0.01

+-------------------------------------------+------------+------------+
| Total wallclock time elapsed since start  |       227s |            |
|                                           |            |            |
| Section                          | no. calls |  wall time | % of total |
+----------------------------------+-----------+------------+------------+
| Assemble Jacobian matrix         |        30 |       147s |        65% |
| Compute residual                 |        41 |      30.4s |        13% |
| Create power                     |        41 |      6.21s |       2.7% |
| Distribute dofs                  |         1 |     0.343s |      0.15% |
| Make grid                        |         1 |      1.47s |      0.65% |
| Output solution                  |         2 |      7.97s |       3.5% |
| Project temperature field        |         1 |   0.00205s |    0.0009% |
| Set environment                  |         1 |  0.000363s |   0.00016% |
| Solve system                     |        30 |      40.5s |        18% |
+----------------------------------+-----------+------------+------------+
```

Figure 7: Mode 1 terminal output for the nonlinear case.

For additional information to be displayed to the screen, the **-verbose** flag can be passed to the variable **$heat_configuration$**. This flag is useful to obtain run-time information of a thermal simulation. A sample of the verbose output is shown in Figure 8. For nonlinear problems this shows the norms of the residuals before and after each Newton step. In addition, the number of iterations required for the conjugate gradient (CG) [18] method to converge for the solution of the linear systems is also displayed.

```
Simulation data:
     Model type nonlinear
     Tmax 0.1
     dt 0.01
     Timestep method Backward Euler
     Element Q1
     Adaptive mesh refinement 0
     Adaptive time integration 0

Initial residual norm || F(u^0,k) || = 0.0343244

Time step 1 at t=0.01 with timestep dt 0.01
Residual norm before Newton step || F(u^n+1,k) || = 0.0343244
     System matrix solved in 13 CG iterations.
Residual norm || F(u^n+1,k+1) || = 0.00141135 at Newton step 1
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 1.65303e-06 at Newton step 2
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 2.77518e-08 at Newton step 3

Time step 2 at t=0.02 with timestep dt 0.01
Residual norm before Newton step || F(u^n+1,k) || = 0.0385185
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 0.000513979 at Newton step 1
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 6.51564e-07 at Newton step 2
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 8.73416e-09 at Newton step 3

Time step 3 at t=0.03 with timestep dt 0.01
Residual norm before Newton step || F(u^n+1,k) || = 0.0298296
     System matrix solved in 13 CG iterations.
Residual norm || F(u^n+1,k+1) || = 0.000625983 at Newton step 1
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 9.11968e-07 at Newton step 2
     System matrix solved in 12 CG iterations.
Residual norm || F(u^n+1,k+1) || = 1.40589e-08 at Newton step 3
```

Figure 8: Mode 1 verbose terminal output for the nonlinear case.

5 Tool Overview

The thermal analysis of an IC requires a description of the physical and material properties of all the constitutive parts of a circuit which are then used to calibrate and execute a thermal simulation. In the MTA, the circuit design and data to calibrate the thermal simulation is encoded in an XML file with a specific format. The format of the XML file and the required simulation data, for circuits at the block and cell-level are described in Section 7. The thermal simulation is performed by numerically solving the heat equation using the FEM and unconditionally stable implicit time stepping methods. The features and mathematical details of the thermal simulator are described in Section 8. The FEM requires a computational mesh representing the circuit structures. In the MTA, this mesh is automatically created by the mesh generator, which is described in Section 9.

A visual representation of the module dependencies in the MTA are shown in Figure 9.

The tool requires an XML file that describes the geometry and material properties of the circuit and

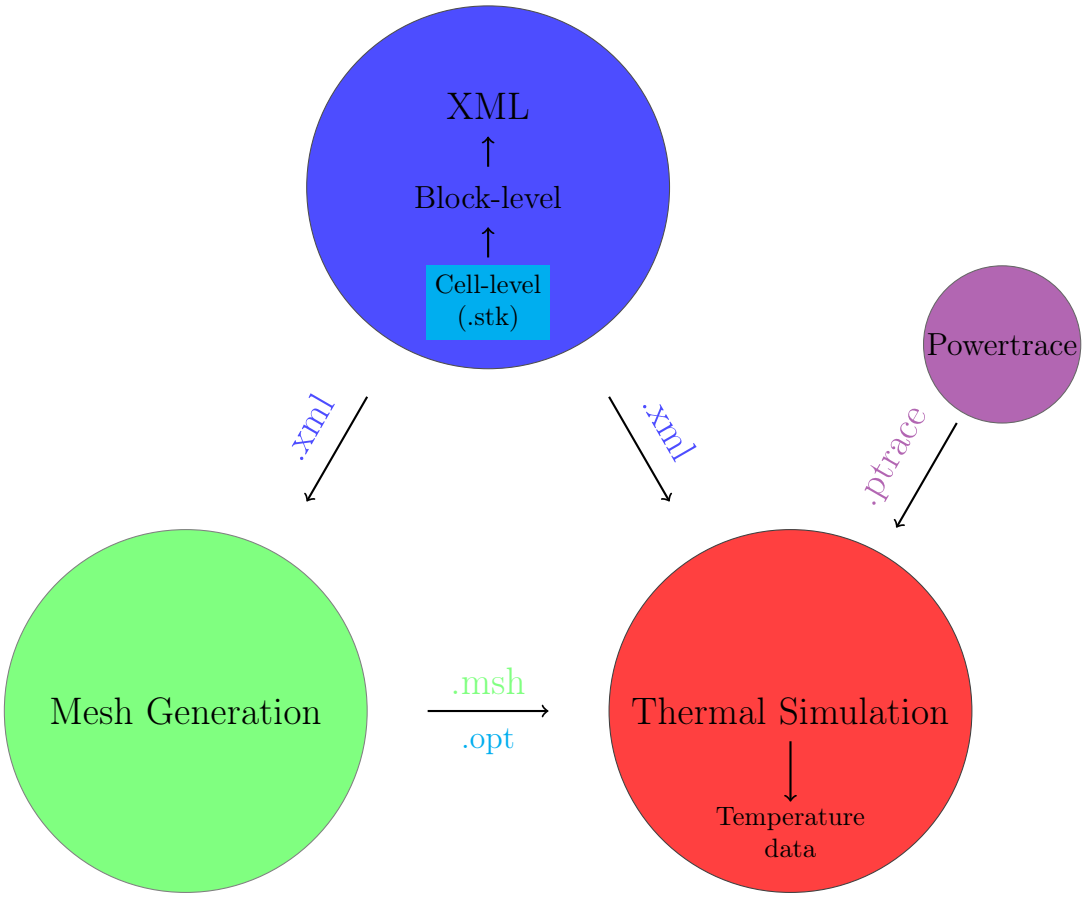


Figure 9: Visual illustration of the MTA tool flow.

package. This module is highlighted in blue. The XML file has a strict hierarchical structure as indicated by the arrows. Block-level assumes an XML file that describes the package and larger functional blocks of the circuit. This file must be generated by the user. The format of the block-level XML file and supported structures is described in Section 7.1. For cell-level thermal simulations, a user defined stack file (highlighted in cyan) must be supplied to the tool which is used to create a die XML file. The stack file contains the dimensions of the die and links to the industrial standard design exchange format (DEF) and library exchange format (LEF) files. These files are parsed by the MTA and create output a die XML file. For cell-level thermal simulations, this die XML file needs to be included inside of the block-level XML file that contains the package information. This process is fully described in Section 7.2. The XML module always outputs an XML file (highlighted in blue) that is passed to both the mesh generation module and thermal simulation module.

The mesh generation module, shown in the light green circle, parses the outputted XML file and creates a finite element mesh (.msh) file that is passed as an input to the thermal simulator. For cell-level thermal simulations, an additional option file (.opt in cyan) used to configured the cell-level mesh reduction algorithms is also passed to the thermal simulator, which is described in Section 7.2.

The thermal simulation module in the red circle, takes as an input the XML file, the mesh (and option) file, and an externally generated powertrace file (.ptrace in light purple). The output of the thermal simulation is the temperature profile of the circuit. The MTA outputs either VTK files, which can then be visualized with an external visualization tool, and a simpler text file format with node coordinates and the associated temperature data.

The execution flow of the tool is controlled with the run flow Perl script `run_flow.pl` which can be customized to suit user's requirements. For instance, to execute a full block level thermal analysis, the user first creates their block level XML file. This XML file is then passed to the mesh generation module which creates the finite element mesh. The XML file, mesh file (.msh), and an externally generated powertrace file (.ptrace) are then supplied to the thermal simulation module, which then computes the temperature profile of the circuit.

The modules can also function independently. For instance, cell-level dies can be automatically generated from standard DEF and LEF files in the XML module and then be integrated inside of a block level XML file containing the package information. Meshes can also be independently created without executing thermal simulations as well. In this way, each of the various aspects of the thermal analysis tool chain can be separately configured to fit a designers requirements. The full range of the MTA's capabilities and how to operate them are explained in the following sections.

6 Run flow script configuration

The execution of the MTA is controlled by the executable Perl script `run_flow.pl`. This script allows users to completely configure and customize their thermal analyses. In the script, the user can specify the required input files, the data output directories, and other directories containing executables for external tools.

To configure the thermal analysis, a user passes flags to the following variables:

$msh_configuration$ configures the creation and refinement of the mesh (Section 9),

$heat_configuration$ configures the thermal simulations (Section 8.2), and

$die_configuration$ configures the creation of the (cell-level) XML file (Section 7.2).

These variables are defined between lines 29-33 in the file `run_flow.pl`. Each of these variables controls a module shown in Figure 9, which allows users to flexibly customize them to suit their requirements. Details of the flags for these variables are given in their respective sections.

To obtain run-time help information and the list of option flags that can be passed to `run_flow.pl` at the command line use the **-help** flag

perl run_flow.pl -help or **perl run_flow.pl -h**.

This command displays a list of option flags to the screen. These flags are specific to the run flow script and are independent from the flags passed to the configuration variables. The full list of run flow flags is given below.

- **-runMESH -mesh <filename.msh>** executes a thermal simulation using an existing mesh, thus bypasing the mesh generation process from the XML file. The **-mesh** flag followed by the filename of the existing mesh must be supplied.
- **-runCELL** execute a cell-level thermal simulation. The block-level XML file that contains package information must alos include a reference to a die XML file. This is explained in more detail in Section 7.2.
- **-mesh <filename.msh>** sets the name of the output mesh file (.msh) to filename.msh.
- **-mode <0-4>** sets the operation mode of the MTA. The modes are described in Section 8.
- **-gmsh** opens the .msh file for visualization in GMSH [15]. The path to GMSH must be assigned in `run_flow.pl`.
- **-paraview** opens the **solution.pvd** file for visualization in Paraview [1]. The path to Paraview must be assigned in `run_flow.pl`.
- **-mpi <N>** executes thermal simulations with MPI on **N** cores. Default is **N**=1.
- **-folder <Output>** specifies the output folder (both the mesh and solution data). Default name is Output.
- **-lcf <lcf_name>** creates an XML file based on the .lcf format of Hotspot 6.0 [19]. Using this flag generates the XML but does not execute a thermal simulation. The user must then re-run the Perl script with the newly created XML file (without the -lcf flag) to execute the thermal simulation. In the example `ev6_HS` you can create an XML from the 3D test case provided by Hotspot 6.0 [19]. This functionality allows compatibilty with Hotspot, i.e., the problem geometry specifications done for Hotspot can be directly downloaded to the MTA.
- **-stack <filename.stk>** specifies the stack file. The stack file contains information such as the DEF and LEF file names that the MTA converts into a die XML file that describes the cell-level die. This is described further in Section 7.2.

The use of these flags depends on the type of thermal analysis the user wishes to perform. More specific details on proper command line syntax for the run flow script and how to, for example, create cell-level die XML files, generate meshes, and execute computationally efficient thermal simulations are given in the following sections. In summary, the run flow Perl script is an interface through which users can customize their thermal analyses.

7 XML Module

The core file required to operate the MTA is the XML file. In this section, detailed information on the structure of this file is presented. This information is crucial for users to understand in order to configure and implement their own designs.

The XML file is structured in a hierarchical fashion. At a coarse level, the user defines large block components of the circuit and within each block further describes the various subcomponents and their associated thermal parameters. In this way, different granularity levels are naturally encoded within the XML file hierarchy. In Figure 10, an abstract hierarchical view of a single tier IC is shown as an XML file.

```
IC
├── Package
│   └── Die-1
│       ├── Active Layer Die-1
│       │   └── DEF-LEF layout Die-1
│       └── Substrate Die-1
└── Heatsink
```

Figure 10: An XML hierarchy of a one tier IC.

This allows users to execute both block- and cell-level thermal simulations. For block-level simulations, users must hand code their XML files. The format and the supported structures for block-level XML files are described in Sections 7.1 and 7.1.1. For cell-level simulations, the MTA automatically generates an XML file that corresponds to the cell-level die based on a stack input file which links to the physical design of the die encoded in the industrial standard design exchange format (DEF) and library exchange format (LEF) files. This die can then be attached to a package by including the die XML file name inside the package XML file. This procedure and the correct syntax to generate the die XML files and execute cell-level thermal simulations is given in Section 7.2.

7.1 Block-level description

Blocks at the floorplan level are considered to be cuboid areas with certain material properties. Different blocks within the XML file are identified by their `<name0>` variable, which is the master name for that block. The definition of each block requires two mandatory delimiters `<parameter>` and `<position>`, and one optional delimiter `<component>` which can be used to define subcomponents of the block.

The thermal parameters of the block, namely, its thermal conductivity κ $[W/mK]$ and volumetric heat capacity c_V $[J/m^3K]$, are specified within the `<parameter>` delimiter. The basic configuration assumes a thermal model where the thermal parameters are constant with respect to temperature, i.e., $\kappa = \kappa_0$ and $c_V = c_0$. The value for κ is specified within the `<ThermalConductivity>` delimiter, and the value of c_V is specified within the `<VolumetricHeatCapacity>` delimiter.

For nonlinear thermal simulations, the MTA assumes temperature dependent thermal parameters of the form $\kappa(u) = \kappa_0 + \kappa_1 u$ and $c_V(u) = c_0 + c_1 u$, where u is the temperature in K. These values can be determined by linearly interpolating the appropriate thermal parameter values listed in [26, 27]. The values for κ_0 and κ_1 are specified within the `<ThermalConductivity>` and `<ThermalConductivityAlpha>` (shown in red in Listing 1) delimiters, respectively. The values for c_0 and c_1 are specified within the `<VolumetricHeatCapacity>`

and `<VolumetricHeatCapacityAlpha>` (shown in red in Listing 1)) delimiters, respectively. An example of the previously described block structure within the XML file is given in Listing 1

Listing 1: Sample XML code for a block

```
<name0>
        <parameter>
                <ThermalConductivity> </ThermalConductivity>
                <ThermalConductivityAlpha></ThermalConductivityAlpha>
                <VolumetricHeatCapacity></VolumetricHeatCapacity>
                <VolumetricHeatCapacityAlpha></VolumetricHeatCapacityAlpha>
        </parameter>
        <position>
                <x></x>
                <y></y>
                <z></z>
                <length></length>
                <width></width>
                <height></height>
        </position>
        <component></component>
</name0>
```

The user must then specify the position of that particular block within the `<position>` delimiter. This is accomplished by specifying the (x, y, z) coordinates of the origin node of that block within the `<x>`, `<y>`, and `<z>` delimiters and the length, width, and height must be specified within the `<length>`, `<width>`, and `<height>` delimiters. The origin node is the node whose (x, y, z) coordinates are simultaneously smaller than or equal to the coordinates of any other node in that block. The position values must be specified in meters.

In Listing 2, a slightly more complex sample XML code is presented, where two subblocks labeled subname1 and subname2 are defined under block **name0**. The specification of the position and parameters of the subblocks is the same as for the original **name0** block. The coordinates as well as the length, width, and height of the component are necessary in order to generate the computational mesh; see Section 9 for further details.

The user must also set the following problem-specific values within the XML file.

- The *atomic time step* is set within the `<TimeStep>` delimiter. This is the base unit of time over which power is dissipated to the circuit and corresponds to the fixed time interval defined by the powertrace file. This based unit of time is determined by the configuration of the power simulator.
- The *ambient temperature*, i.e., the temperature of the external enviroment surrounding the circuit is a constant value set within the `<AmbientTemperature>` delimiter.
- The *initial temperature* of the circuit is set to a constant value within the `<InitialTemperature>` delimiter.
- The *heat transfer coefficients* at the bottom, side, and top boundary walls of the circuit are set to the values specified within the `<HeatTransferCoefficientBottom>`, `<HeatTransferCoefficientSide>`, `<HeatTransferCoefficientTop>` delimiters, respectively. These coefficients are to set Robin boundary conditions (Section 8.1.2). The user can specify a single heat transfer coefficient using the delimiter `<HeatTransferCoefficient>`. These values must be specified in order to set the appropriate heat flux boundary conditions.

These problem specific values are provided as parameters to the component of the highest level of the hierarchy. For the example shown in Listing 2, this corresponds to the block **name0**.

Listing 2: XML example including functional subblocks

```
<name0>
        <component>
                <subname1>
                        <component>
                                <subname3>
                                        <component></component>
                                        <position></position>
                                        <parameter></parameter>
                                </subname3>
                        </component>
                        <position></position>
                        <parameter></parameter>
                </subname1>
                <subname2>
                        <component></component>
                        <position></position>
                        <parameter></parameter>
                </subname2>
        </component>
        <position></position>
        <parameter>
                <TimeStep></TimeStep>
                <AmbientTemperature></AmbientTemperature>
                <InitialTemperature></InitialTemperature>
                <HeatTransferCoefficientTop></HeatTransferCoefficientTop>
                <HeatTransferCoefficientBottom></HeatTransferCoefficientBottom>
                <HeatTransferCoefficientSide></HeatTransferCoefficientSide>
        </parameter>
</name0>
```

More detailed sample XML files are included in the XML gallery located on our website:

https://staffnet.cs.manchester.ac.uk/acso/thermal-analyzer/.

7.1.1 Supported geometries

The MTA supports cuboids (hexahedrons) as the basic component for constructing the computational domain of the IC. The mesh generator reads from the geometric definitions in the XML file and creates the computational mesh. The MTA generates three basic types of cuboid: `Cuboid`, `CuboidArray`, and `HeatSink`.

The type “Cuboid" is the base hexahedral structure; see Figure 11. It is considered as a default setting, which does not need to be specified explicitly. Instead only six parameters must be specified, the 3 coordinates and length, width, and height.

The type “CuboidArray" defines an array of cuboids simultaneously; see Figure 12. Each cuboid shares the same component name and the same physical parameters. In this case the type needs to be explicitly set to type=“CuboidArray" in the XML file. The parameters of the CuboidArray are depicted in Figure 12.

```
<x>: origin x-coordinate of cuboid
<y>: origin y-coordinate of cuboid
<z>: origin z-coordinate of cuboid
<length>: size of cuboid in x-axis
<width>: size of cuboid in y-axis
<height>: size of cuboid in z-axis
```

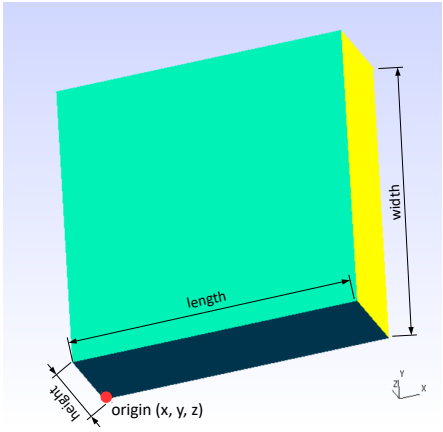


Figure 11: Cuboid structure (right) and its XML code (left).

```
<length>: size of a cuboid in the x-direction
<width>: size of a cuboid in the y-direction
<xstart>: center x-coordinate of first cuboid
<ystart>: center y-coordinate of first cuboid
<xnumber>: number of cuboids in the  x-direction
<ynumber>: number of cuboids in the y-direction
<xend>: center x-coordinate of last cuboid
<yend>: center y-coordinate of last cuboid
<z>: bottom (origin) for the cuboids
<height>: height of all cuboids in th z-direction
```

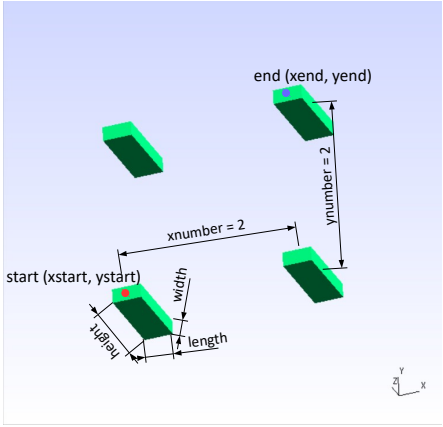


Figure 12: Cuboid array structure (right) and XML code (left).

The type “HeatSink" is a specific type used to define the surface of a heatsink; see Figure 13. The “HeatSink" type is automatically defined to be the top boundary of the circuit domain. This top boundary is associated with the top heat transfer coefficient (e.g., `<HeatTransferCoefficientTop>`) in the simulation. The geometry of the heatsink has two major parts, the fin structure on top and the supporting cuboids at the bottom of the fins. The definition of the fins is similar to the type “CuboidArray" while the supporting bottom cuboids are similar to the type “Cuboid".

```
<xCuboid>: x-coordinate of origin of bottom cuboid
<yCuboid>: y-coordinate of origin of bottom cuboid
<zCuboid>: z-coordinate of origin of bottom cuboid
<lengthCuboid>: size of cuboid in x-direction
<widthCuboid>: size of cuboid in y-direction
<heightCuboid>: size of cuboid in z-direction
<lengthFin>: size of each fin in x-direction
<widthFin>: size of each fin in y-direction
<heightFin>: size of each fin in z-direction
<xstartFin>: center x-coordinate of first cuboid
<ystartFin>: center y-coordinate of first cuboid
<xnumberFin>: number of fins in x-direction
<ynumberFin>: number of fins in y-direction
<xendFin>: center x-coordinate of last cuboid
<yendFin>: center y-coordinate of last cuboid
```

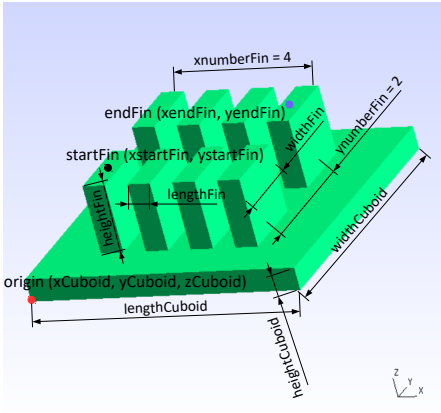


Figure 13: Heat sink structure (right) and XML code (left).

7.2 Cell-level description

The MTA automates the generation of XML files that represent cell-level circuits. This requires defining a stack (.stk) file that defines the active and substrate layers of a die. An example of this file is displayed in Table 1.

#LayerID	#X	#Y	#Z	#Length	#Width	#Height	#k	#cv	#LEF(.lef)	#DEF(.def)
activelayer	0.0	0.0	2e-5	NA	NA	6.095e-6	100	1.75e6	lef_filename.lef	def_filename.def
substrate	0.0	0.0	0.0	6.8654e-04	6.84e-04	2e-5	100	1.75e6		

Table 1: Stack file example

The user specifies the (x, y, z)-coordinates of the footpoints for both the active layer and substrate, in addition to the length and width. For the active layer where the placed cells are, the length and width are provided in the DEF and LEF files. These file names are specified under the LEF and DEF columns in Table 1. The user must also define the thermal conductivity k and the specific heat c_V. To create the cell-level die, the user executes the command

perl run_flow.pl -xml die.xml -stack filename.stk

which produces as an output the cell-level die.xml file. To attach the die to a specific package, the user must

have a predefined XML file of a corresponding package. The die is then an input file in the package XML. How to include this file is shown in Listing 3, where the input die.xml file is highlighted in cyan.

Listing 3: Package XML file with die XML included

```
<IC>
    <component>
        <tier0 file="die.xml"></tier0>
        <HeatSink type="HeatSink">
            <component></component>
            <parameter>
                    <ThermalConductivity>></ThermalConductivity>
                    <VolumetricHeatCapacity></VolumetricHeatCapacity>
            </parameter>
            <position>
                <xCuboid>-</xCuboid>
                <yCuboid>-</yCuboid>
                <zCuboid></zCuboid>
                <lengthCuboid></lengthCuboid>
                <widthCuboid></widthCuboid>
                <heightCuboid></heightCuboid>
                <lengthFin></lengthFin>
                <widthFin></widthFin>
                <heightFin></heightFin>
                <xstartFin></xstartFin>
                <ystartFin></ystartFin>
                <xnumberFin></xnumberFin>
                <ynumberFin></ynumberFin>
                <xendFin></xendFin>
                <yendFin></yendFin>
            </position>
        </HeatSink>
    </component>
    <parameter>
        <ThermalConductivity> </ThermalConductivity>
        <VolumetricHeatCapacity> </VolumetricHeatCapacity>
        <HeatTransferCoefficient> </HeatTransferCoefficient>
        <AmbientTemperature> </AmbientTemperature>
        <InitialTemperature> </InitialTemperature>
        <TimeStep> </TimeStep>
    </parameter>
    <position>
        <x> </x>
        <y></y>
        <z></z>
        <length></length>
        <width></width>
        <height></height>
    </position>
</IC>
```

Once the die XML file has been included in the package XML, the corresponding mesh can be generated and a cell-level thermal simulation can be executed. The meshes produced by such cell-level dies can be very fine and contain a large number of nodes. The MTA offers homogenization techniques to reduce the size of the meshes and thus the size of the discrete problems that need to be handled during the simulation. These techniques are described in Section 9.

8 Thermal simulation module

Thermal simulations within the MTA are performed by numerically solving the heat equation, the partial differential equation (PDE) governing the flow of heat within a material body [13, 14]. This is achieved by discretizing the problem in space using the finite element method (FEM) [12, 16] and in time with unconditionally stable implicit time stepping methods [23]. The numerical schemes within the MTA are implemented using the open-source FEM library `deal.II` [4, 5]. The MTA has the ability to execute thermal simulations in parallel on multicore architectures.

The MTA is designed to operate in the following modes:

Mode 0 Steady-state thermal analysis on a fixed grid,

Mode 1 Transient thermal analysis with a fixed grid and fixed time step size,

Mode 2 Transient thermal analysis with a fixed grid and adaptive time step size,

Mode 3 Transient thermal analysis with an adaptively refined grid and fixed time step size,

Mode 4 Transient thermal analysis with simultaneous adaptive spatiotemporal refinement.

We first review the mathematical details behind the methodology of these modes and give details on the corresponding numerical methods. We then describe the thermal configuration flags to calibrate the thermal simulation.

8.1 Mathematical details

To facilitate a better understanding of the underlying mathematics and the numerical solution of the heat equation in the MTA, we provide an overview of the governing equations and their discretization for both steady-state and transient thermal analyses. This overview is important because it provides details on the different heat flow models (such as linear or nonlinear) in the MTA and introduces the important parameters a user can adjust in their simulations.

8.1.1 The Poisson equation

The steady-state temperature distribution $u(\mathbf{x})$ at a spatial point $\mathbf{x} = (x, y, z)$ in a (cuboid) material body $\Omega \subset \mathbb{R}^3$ is governed by the boundary value problem

$$-\nabla \cdot (\kappa(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}), \text{ in } \Omega, \tag{8.1a}$$

$$\kappa(\mathbf{x})\nabla u(\mathbf{x}) \cdot \mathbf{n} = g(\mathbf{x}), \text{ on } \partial\Omega_N, \tag{8.1b}$$

$$\kappa(\mathbf{x})\nabla u(\mathbf{x}) \cdot \mathbf{n} = \eta(u_a - u(\mathbf{x})), \text{ on } \partial\Omega_R. \tag{8.1c}$$

This boundary value problem is commonly referred to as the Poisson problem. The nabla symbols in (8.1a) are the divergence and gradient operators, denoted by $\nabla \cdot (u(\mathbf{x})) = u_x + u_y + u_z$ and $\nabla u(\mathbf{x}) = (u_x, u_y, u_z)^T$,

respectively. The boundary conditions consist of Neumann (zero flux) conditions on $\partial\Omega_N$ and Robin conditions (flux proportional to the ambient temperature difference, i.e., Newton's law of cooling) on $\partial\Omega_R$ and the boundaries $\partial\Omega_N$ and $\partial\Omega_R$ do not overlap, i.e., $\partial\Omega_N \cap \partial\Omega_R = \emptyset$. The outward unit normal to the boundary $\partial\Omega = \partial\Omega_N \cup \partial\Omega_R$ is denoted by $\mathbf{n}$. The thermal parameters are the thermal conductivity $\kappa(\mathbf{x})$, and the thermal transmissivity η. Note that the thermal conductivity $\kappa(\mathbf{x})$ has spatial dependence since the various components of the circuit can have different thermal conductivities.

The dimensional quantities of the variables in the Poisson equation are the temperature $u(\mathbf{x})$ in units of Kelvin [K], the power density $f(\mathbf{x})$ in units Watts per meter-cubed $[W/m^3]$ which is computed by dividing the discrete values in the powertrace file with the appropriate volume, the thermal conductivity in Watts per meter-Kelvin $[W/mK]$, and the thermal transmissivity η in Watts per meter squared-Kelvin $[W/m^2K]$. Note that $g(\mathbf{x})$ has units $[W/m^2]$. In the simplest case, the parameters κ and η are temperature-independent which yields a linear heat flow model.

The boundary value problem (8.1) is solved numerically using the finite element method (FEM). The readers not familiar with the FEM are is referred to, e.g., [9, 12, 16] for complete mathematical and computational details. The FEM initially partitions the domain into a union of non-overlapping elements $\Omega = \cup_k \Omega_k$. The elements Ω_k are typically tetra or hexahedra (cuboids) with a characteristic length h called the mesh width. The finite element solution is obtained by interpolating the computed discrete nodal values at the vertices of the elements, i.e., the solution has the form $u_h(\mathbf{x}) = \sum_{j=1}^{n} u_j \phi_j(\mathbf{x})$, where $u_j \approx u(\mathbf{x}_j)$ are the n unknown discrete nodal temperature values, and $\phi_j(\mathbf{x})$ is a global Lagrangian basis function that equals 1 at node $\mathbf{x}_j$ and zero at every other node in the mesh. Typically, the basis functions are low-order polynomials defined on a patch of elements adjacent to a node $\mathbf{x}_j$. For an asymptotic error order of 1 in 3-D, these basis functions are trilinear and have the form $\phi(x, y, z) = c_0 + c_1 x + c_2 y + c_3 z + c_4 xy + c_5 xz + c_6 yz$, and are referred to as Q1 elements. For an asymptotic error order of 2 in 3-D, the basis functions are triquadratic and referred to as Q2 elements. The user can change the order of the elements in the simulation with the optional **-order** flag as described in Section 8.2.1. The n DOFs or unknown nodal values $\{u_j\}_{j=1}^{n}$, can be collected into a column vector $\boldsymbol{u}_h = [u_1, \ldots, u_n]^T$. This vector of unknowns is then determined by solving a linear system of n equations.

For the steady-state heat equation, the discretization of the heat equation using the FEM leads to the following system of equations

$$K\mathbf{u}_h = \mathbf{f}, \tag{8.2}$$

where $K_{i,j} = \int_\Omega \kappa(\mathbf{x}) \nabla\phi_j \nabla\phi_i \; d\mathbf{x} + \eta \int_{\partial\Omega_R} \phi_j \phi_i \; dS$ is the (symmetric) stiffness matrix ($K \in \mathbb{R}^{n\times n}$), and $\mathbf{f}_i = \int_\Omega f\phi_i \; d\mathbf{x} + \int_{\partial\Omega_N} g(x)\phi_i \; d\mathbf{x} + \int_{\partial\Omega_R} u_a \phi_i \; d\mathbf{x}$ is the right hand side source function vector assembled from the discrete power values in the powertrace file and the contributions from the boundary conditions.

The steady state temperature approximation $\mathbf{u}_h$ is obtained by solving (8.2). The default solver in the MTA is the conjugate gradient (CG) algorithm with an algebraic multigrid (AMG) preconditioner. This methodology has a computational cost that scales linearly in the number of unknowns in the system and is known to be scalable on modern parallel architectures [6, 7, 18, 28]. The implementation of the iterative solver is accomplished within deal.II [4, 5] by making calls to the open-source scientific computing library PETSc [2, 3]. The CG method terminates when the norm of the relative residual is below a user-prescribed tolerance. In the MTA, the default CG solver tolerance is set to 10^{-8}.

8.1.2 The linear heat equation

The time evolution of the temperature $u(\mathbf{x}, t)$ in a material body $\Omega \subset \mathbb{R}^3$ is governed by the initial boundary value problem

$$c_V(\mathbf{x})\frac{\partial}{\partial t}u(\mathbf{x}, t) - \nabla \cdot (\kappa(\mathbf{x})\nabla u(\mathbf{x}, t)) = f(\mathbf{x}, t), \text{ in } \Omega \times [0, T], \tag{8.3a}$$

$$\kappa\nabla u(\mathbf{x}, t) \cdot \mathbf{n} = g(\mathbf{x}, t), \text{ on } \partial\Omega_N \times [0, T], \tag{8.3b}$$

$$\kappa\nabla u(\mathbf{x}, t) \cdot \mathbf{n} = \eta(u_a - u(\mathbf{x}, t)), \text{ on } \partial\Omega_R \times [0, T], \tag{8.3c}$$

$$u(\mathbf{x}, 0) = u_0(\mathbf{x}), \text{ in } \Omega. \tag{8.3d}$$

The only new term is the partial derivative of the temperature with respect to time $\frac{\partial}{\partial t}u(\mathbf{x}, t)$, which represents the acceleration term. Note that time-dependent thermal simulations require an initial condition $u_0(\mathbf{x})$, which is the starting temperature of the circuit. The only other new thermal parameter is the spatially-varying volumetric heat capacity $c_V(\mathbf{x})$ $[J/m^3K]$. Note that the thermal parameters in the initial boundary value problem (8.3) are temperature-independent, yielding a linear thermal model.

The spatial discretization of the linear heat equation using the FEM results in the following system of initial value problems (IVPs)

$$M\dot{\mathbf{u}}_h + K\mathbf{u}_h = \mathbf{f}, \tag{8.4}$$

where, as before, $K_{i,j} = \int_\Omega \kappa(\mathbf{x})\nabla\phi_j\nabla\phi_i \; d\mathbf{x} + \eta\int_{\partial\Omega_R} \phi_j\phi_i \; dS$ is the stiffness matrix ($K \in \mathbb{R}^{n\times n}$) and $\mathbf{f}_i = \int_\Omega f\phi_i \; d\mathbf{x} + \int_{\partial\Omega_N} g(x)\phi_i \; d\mathbf{x} + \int_{\partial\Omega_R} u_a\phi_i \; d\mathbf{x}$ is the right hand side source function vector. The new mass matrix is $M_{i,j} = \int_\Omega c_V(\mathbf{x})\phi_j\phi_i \; d\mathbf{x}$ ($M \in \mathbb{R}^{n\times n}$) and $\dot{\mathbf{u}}_h$ denotes the time derivative of the finite element function.

To discretize (8.4) in time, the MTA uses unconditionally stable implicit time integration methods. For the simplest case where the time step size is fixed to the atomic time step Δt_a (determined by the powertrace file), the time interval $[0, T]$ is partitioned into N_a subintervals such that $t_0 = 0$ and $t_k = k\Delta t_a$ for $k = 1, \ldots, N_a$, with $\Delta t_a = T/N_a$. The discrete solution and right hand side vector at time t_k are denoted by $\mathbf{u}_h^k$ and $\mathbf{f}_h^k$, respectively. Larger time step sizes that are integer multiples of the atomic time step size can also be taken.

Implicit time integration methods work by replacing the time derivative in (8.4) by a finite difference approximation, i.e., $\dot{\mathbf{u}}_h \approx D\mathbf{u}_h^{k+1}$, for some difference operator D. Then the following equation is solved for $\mathbf{u}_h^{k+1}$

$$M\left(D\mathbf{u}_h^{k+1}\right) + K\mathbf{u}_h^{k+1} = \mathbf{f}_h^{k+1}. \tag{8.5}$$

The MTA supports the following time stepping methods:

- the 1st order backward Euler (BE) method,
- the 2nd order backward differentiation formula (BDF2),
- the 2nd order trapezoidal rule (TR) and the stabilized version of this method [22],
- the 2nd order implicit midpoint rule (IMR).

For an overview of these methods, see, e.g., [23].

The BE method is obtained by setting the difference operator to $D_1\mathbf{u}_h^{k+1} = \frac{\mathbf{u}_h^{k+1} - \mathbf{u}_h^k}{\Delta t}$. In this case (8.5) becomes

$$(M + \Delta t K)\mathbf{u}_h^{k+1} = M\mathbf{u}_h^k + \Delta t\mathbf{f}_h^{k+1}. \tag{8.6}$$

The BDF2 method is obtained by setting $D_2\mathbf{u}_h^{k+1} = \frac{3\mathbf{u}_h^{k+1}-4\mathbf{u}_h^{k}-\mathbf{u}_h^{k-1}}{2\Delta t}$ for which a similar expression to (8.6) can be derived. The trapezoid rule is defined by

$$M\left(D_1\mathbf{u}_h^{k+1}\right) + \frac{1}{2}\left(K\mathbf{u}_h^{k+1} - \mathbf{f}_h^{k+1} + K\mathbf{u}_h^{k} - \mathbf{f}_h^{k}\right) = 0. \tag{8.7}$$

which translates into solving a linear system

$$\left(M + \frac{\Delta t}{2}K\right)\mathbf{u}_h^{k+1} = \left(M + \frac{\Delta t}{2}K\right)\mathbf{u}_h^{k} + \frac{\Delta t}{2}\left(\mathbf{f}_h^{k+1} + \mathbf{f}_h^{k}\right). \tag{8.8}$$

The stabilized trapezoid rule replaces $K\mathbf{u}_h^{k+1} - \mathbf{f}_h^{k+1}$ in (8.7) every k^* time steps with the finite difference interrupt (FDI) $D_2\mathbf{u}_h^{k}$. Details on how to set k^* are given in [12, 22]. For linear problems IMR is equivalent to the TR method. The IMR method is important for designing energy conserving numerical schemes (the so-called geometric integrators) and is discussed further in Section 8.1.4 below.

Regardless of the time integration method, computing the fully discretized temperature $\mathbf{u}_h^{k+1}$ at time t_{k+1} requires the solution of a linear system of the form

$$A^{k+1}\mathbf{u}_h^{k+1} = \tilde{\mathbf{f}}^{k+1}, \tag{8.9}$$

where A^{k+1} is an appropriate combination of the matrices M and K, and $\tilde{\mathbf{f}}^{k+1}$ is the corresponding right hand side vector. Note that in Mode 1, $A^{k+1} = A^1$, i.e., the system matrix remains the same throughout the computation.

The major costs in computing the temperature profile are the assembly of the linear system and its solution. In Mode 1, where the finite element mesh and time step size are fixed throughout the computation, the linear system needs to be assembled only once at the first time step. Although the vector $\tilde{\mathbf{f}}_h^{k}$ must be computed at each time step, the cost of its assembly is significantly smaller than the cost of assembling a sparse system matrix.

Thus, the main cost per iteration (excluding the assembly at the first time step) is the solution of the linear system. As previously mentioned (cf. Section 8.1.1), the default solver in the MTA is the CG algorithm with an AMG preconditioner. For large-scale problems, especially those arising from the discretization of time-dependent 3-D PDEs, preconditioned iterative methods yield superior performance compared to sparse direct solvers, such as SuperLU [24], both in terms of memory consumption and execution time [21]. Moreover, since the matrices are the same at each time step, the preconditioner only needs to be setup at the first step and can be re-used afterwards, making for a more efficient implementation.

8.1.3 Adaptive spatiotemporal refinement

In this section we present some of the mathematical details pertaining to the adaptive spatiotemporal refinement features of the MTA. Users can enable these adaptive refinement features individually, such as with Modes 2 and 3 (for adaptive temporal and spatial refinement, respectively), or simultaneously in Mode 4. The computational details for the adaptive spatiotemporal refinement are given in the corresponding subsections of Section 8.2.

Adaptive mesh refinement The temperature can vary significantly across the area of a circuit due to highly localized and time-dependent heat sources and the different properties of the constituent materials. Capturing this behavior accurately while maintaining the overall computational efficiency requires locally

fine meshes. To achieve the required level of accuracy with initially created uniform meshes would lead to unnecessarily large computational times. In order to minimize the computational overhead without sacrificing accuracy, the mesh is adaptively refined only in parts of the domain with significant variations (large gradients) in temperature. Thus, adaptively refined solutions achieve the same asymptotic level of accuracy with more economical spatial discretizations. The adaptive mesh refinement (AMR) strategy in the MTA is based on computed a-posteriori error estimates.

The elements of the computational mesh are marked for refinement based on computed error estimates of the solution. In the MTA, these estimates are computed by the error estimator [20]. This estimator computes an approximation of the error in each element by averaging the jumps in the solution gradients at the faces of the adjacent elements. The refinement/merging of the elements is triggered based on user-specified parameters. For AMR, the key parameters are the AMR frequency, and the percentage of elements with the largest/smallest error estimates to be refined/merged, denoted by τ_r and τ_m, respectively. The parameters $0 < \tau_r, \tau_m < 1$ influence how many elements are marked for refinement/merging and thus control the reduction of the error in the adaptively refined solution and also the computational cost. The refinement strategy is based on Dörfler marking [10]. The set of elements to be refined in the mesh is the smallest subset of hexahedrons whose error sum is larger than the product of τ_r and the total error estimate. The set of elements to be merged is the smallest subset of hexahedrons whose error sum is smaller than the product of τ_m and the total error. Uniform refinement with no merging is achieved by setting $\tau_r = 1$ and $\tau_m = 0$. These tolerances and the flags to enable and specify their values is described in Section 8.2.4.

Adaptive time integration Adaptive time stepping is important for increasing the computational efficiency of transient thermal simulations whilst controlling the temporal error. In these methods, the time step size Δt is determined based on the computed local truncation error (LTE) estimate of the implicit method. This is accomplished using predictor-corrector methods [17]. These methods compute an LTE estimate to sufficient asymptotic accuracy based on the computed predictor (explicit method) solution and corrector (implicit method) solution. The asymptotic order of convergence for both of these methods should be equal. Let $\mathbf{e}$ denote the local truncation error (LTE) estimate [23] and ε_t a user-prescribed tolerance. Then, a standard heuristic for computing the adaptive time step Δt_{n+1} of a method of order q is

$$\Delta t_{n+1} = \Delta t_n \left(\frac{\varepsilon_t}{\|\mathbf{e}\|_2} \right)^{\frac{1}{q+1}}, \tag{8.10}$$

where $\|\mathbf{e}\|_2$ is the standard Euclidean norm and Δt_n is the previous time step. The MTA further adjusts the estimated time step size Δt_{n+1} so that it is a multiple integer of the atomic time step size. In this way the time step is adapted to the local features of the solution. Each time integration method implemented in the MTA supports adaptive time step selection. Enabling this feature allows for larger time steps to be taken as the solution approaches its steady state or when the changes in the solution are not significant. Use of this feature is encouraged for long thermal simulations as the ability to take large time steps can significantly improve simulation times. How to enable the adaptive time stepping routines and set the error tolerance ε_t is described in Section 8.2.3.

8.1.4 The nonlinear heat equation

The nonlinear initial boundary value problem

$$c_V(\mathbf{x}, u)\frac{\partial u(\mathbf{x}, t)}{\partial t} - \nabla \cdot (\kappa(\mathbf{x}, u)\nabla u(\mathbf{x}, t)) = f(\mathbf{x}, t) \text{ in } \Omega \times [0, T], \tag{8.11a}$$

$$(\kappa(\mathbf{x}, u)\nabla u) \cdot \mathbf{n} = \eta(u_a - u) \text{ on } \partial\Omega \times [0, T], \tag{8.11b}$$

$$u(\mathbf{x}, 0) = u_0 \text{ in } \bar{\Omega} = \Omega \cup \partial\Omega, \tag{8.11c}$$

models the flow of heat in an IC with temperature- (and space-) dependent thermal parameters $c_V(\mathbf{x}, u)$ and $\kappa(\mathbf{x}, u)$. Note that the nonlinear steady-state equation is obtained by setting $\frac{\partial u(\mathbf{x},t)}{\partial t} = 0$.

The FEM discretization of (8.11) results in a system of nonlinear IVPs of the form

$$M(\boldsymbol{u}_h)\dot{\boldsymbol{u}}_h + K(\boldsymbol{u}_h)\boldsymbol{u}_h = \mathbf{f}_h, \tag{8.12}$$

or, equivalently, in the residual form as

$$M(\boldsymbol{u}_h)\dot{\boldsymbol{u}}_h + \underbrace{K(\boldsymbol{u}_h)\boldsymbol{u}_h - \mathbf{f}_h}_{F(\boldsymbol{u}_h)} = 0. \tag{8.13}$$

The right hand side source $\mathbf{f}_h$ is the same as for the linear case. Note that the mass matrix and stiffness matrix in (8.13) now depend on the solution $\mathbf{u}_h$, i.e.,

$$M(\mathbf{u}_h)_{ij} = \int_\Omega c_V(\mathbf{x}, \mathbf{u}_h)\phi_j\phi_i \; d\mathbf{x}, \tag{8.14}$$

$$K(\mathbf{u}_h)_{ij} = \int_\Omega \kappa(\mathbf{x}, \mathbf{u}_h)\nabla\phi_j\nabla\phi_i \; d\mathbf{x} + \eta\int_{\partial\Omega} \phi_j\phi_i \; dS \tag{8.15}$$

Similar to the linear case, (8.13) is solved using (adapative) implicit time integration methods. For the nonlinear case the equations have the form

$$M\left(\mathbf{u}_h^{k+1}\right) D\mathbf{u}_h^{k+1} + F\left(\mathbf{u}_h^{k+1}\right) = 0. \tag{8.16}$$

The MTA supports the following time integration methods in nonlinear simulations:

- the 1st order backward Euler (BE) method,
- the 2nd order backward differentiation formula (BDF2),
- the 2nd order implicit midpoint rule (IMR).

The IMR method is obtained by solving

$$M\left(\mathbf{u}_h^{k+1}\right) D_1\mathbf{u}_h^{k+1} + F\left(\frac{1}{2}\left(\mathbf{u}_h^{k+1} + \mathbf{u}_h^{k}\right)\right) = 0. \tag{8.17}$$

At each time step t_{k+1}, a system of nonlinear algebraic equations (8.16) is solved with Newton's method. Newton's method starts from the initial guess $\mathbf{u}_h^{k+1,0}$, which is set to the initial temperature by default, and

computes the approximate solution $\mathbf{u}_h^{k+1,\ell}$ for $\ell = 1, 2...$ by solving the linear system (8.18) for the correction $\delta\mathbf{u}_h^{k+1,\ell-1}$ and updating the solution via (8.19)

$$J\left(\mathbf{u}_h^{k+1,\ell-1}\right)\delta\mathbf{u}_h^{k+1,\ell-1} = -\mathbf{r}\left(\mathbf{u}_h^{k+1,\ell-1}\right), \tag{8.18}$$

$$\mathbf{u}_h^{k+1,\ell} = \mathbf{u}_h^{k+1,\ell-1} + \delta\mathbf{u}_h^{k+1,\ell-1}, \tag{8.19}$$

where

$$J\left(\mathbf{u}_h^{k+1}\right) = \frac{\partial\mathbf{r}\left(\mathbf{u}_h^{k+1}\right)}{\partial\mathbf{u}_h^{k+1}}, \tag{8.20}$$

$$\mathbf{r}\left(\mathbf{u}_h^{k+1}\right) = M\left(\mathbf{u}_h^{k+1}\right)D\mathbf{u}_h^{k+1} + F\left(\mathbf{u}_h^{k+1}\right), \tag{8.21}$$

are the Jacobian matrix and the residual vector at iteration $\ell - 1$, respectively. Note that in (8.20)-(8.21) the Newton iteration index ℓ is dropped for notational convenience. By computing the partial derivatives in (8.20), and using the mass and stiffness matrices from (8.14) and (8.15), the Jacobian and residual are equivalently expressed as

$$\begin{aligned} J_{ij} = &\int_\Omega c_V'(\mathbf{u}_h^{k+1})D\mathbf{u}_h^{k+1}\phi_j\phi_i\ d\mathbf{x} + \int_\Omega c_V(\mathbf{u}_h^{k+1})\phi_j\phi_i\ d\mathbf{x} \\ &+ \int_\Omega \kappa'(\mathbf{u}_h^{k+1})\phi_j\nabla\mathbf{u}_h^{k+1}\nabla\phi_i\ d\mathbf{x} + \int_\Omega \kappa(\mathbf{u}_h^{k+1})\phi_j\nabla\mathbf{u}_h^{k+1}\phi_i\ d\mathbf{x} \end{aligned} \tag{8.22}$$

$$\mathbf{r}_i = \int_\Omega c_V(\mathbf{u}_h^{k+1})D\mathbf{u}_h^{k+1}\phi_i\ d\mathbf{x} + \int_\Omega \kappa(\mathbf{u}_h^{k+1})\nabla\mathbf{u}_h^{k+1}\nabla\phi_i\ d\mathbf{x} + \eta\int_{\partial\Omega_R}(\mathbf{u}_h^{k+1} - u_a)\phi_i\ d\mathbf{x} - \int_\Omega f^{k+1}\phi_i\ d\mathbf{x}. \tag{8.23}$$

This is the exact version of Newton's method which requires the assembly of the Jacobian matrix at each iteration. The stopping criterion for the Newton iteration adopted in the MTA is satisfied when the relative norm of the residual in (8.21) is less than a user-prescribed tolerance ε_N, which is set to 10^{-6} by default. Larger values of ε_N lead to faster simulations with fewer Newton iterations required but less accurate solutions. For linear problems, Newton's method converges in a single iteration.

8.1.5 The linearized heat model

To avoid the computational cost of Newton's method associated with the repeated assembly and solution of the linear systems in (8.18), the nonlinear equations can be considered in linearized form. This is achieved by replacing $\mathbf{u}_h^{k+1}$ in (8.16) with $\bar{\mathbf{u}}_h$ that is a suitable linear combination of the previous history values with an aim of preserving the asymptotic order of convergence. Two examples of such combinations are taking $\bar{\mathbf{u}}_h = \mathbf{u}_h^k$, where only a single previous history value is used, or a higher order approximation $\bar{\mathbf{u}}_h = (1 + \Delta t_k/Deltat_{k-1})\left(\mathbf{u}_h^k + (\Delta t_k/Deltat_{k-1})\mathbf{u}_h^{k-1}\right)$, where a linear combination of the two previous history values are used.

The fully discrete linearized problem thus computes $\mathbf{u}_h^{k+1}$ by solving the system of equations

$$M(\bar{\mathbf{u}}_h)D\mathbf{u}_h^{k+1} + K(\bar{\mathbf{u}}_h)\mathbf{u}_h^{k+1} - \mathbf{f}_h^{k+1} = 0. \tag{8.24}$$

The linearized problem requires only one system assembly per time step, which significantly improves the computational time of the thermal simulation compared to a fully nonlinear simulation while retaining the accuracy. The MTA supports linearized thermal simulations with each of the previously mentioned implicit time steps method.

8.2 Thermal configuration flags

In this section we describe mode specific option flags aimed at the customization of thermal simulations in the MTA. The first five subsections correspond to flags that are relevant to Modes 0-4. The following subsections pertain to flags that can be used in each of the modes, such as how to change the heat flow model from linear to nonlinear and how to set the output frequency of the data.

8.2.1 Mode 0 flags

The MTA computes steady-state solutions of the heat equation by solving the Poisson equation. A brief review of the finite element solution of the Poisson equation is given in Section 8.1.1. Below is a list of optional user-specified thermal configuration flags allowed in this mode. These flags should be set in the **$heat_configuration$** variable in the Perl script `run_flow.pl` as described in Section 6.

- **-order** <**N**> sets the order of the finite element basis functions. The default is $\mathbf{N} = 1$, i.e., trilinear basis functions, or Q1 elements [12]. The allowed set of values is $N = 1, 2$, where 2 corresponds to triquadratic basis functions or Q2 elements.
- **-folder** <**folder_name**> specifies the name of the folder where the output files are directed to. If no folder is specified the MTA creates the directory `Output` and writes solution data to this directory.
- **-verbose** displays extended information regarding the thermal simulation.
- **-output_precision** <**value**> sets the precision of the solution output. The default **value** = 6, i.e., output 6 digits of the solution.
- **-solver_tolerance** <**value**> sets the CG solver tolerance to value. The default **value** = 10^{-8}. A reasonable range of tolerance values is $[10^{-6}, 10^{-10}]$ but can be adjusted depending on the accuracy requirements of the problem.

8.2.2 Mode 1 flags

Recall that for thermal simulations in Mode 1, the problem is discretized in space using a fixed grid and the resulting system of IVPs is solved by an implicit time integration method with a fixed time step size. The discretization details are described in Section 8.1.2. Below we list the option flags to configure Mode 1 thermal simulations.

- **-order** <**N**> sets the order of the finite element basis functions. The default is $\mathbf{N} = 1$. The MTA expects $N = 1, 2$.
- **-folder** <**folder_name**> specifies the name of the folder to direct output to. If no folder is specified the MTA creates the directory `Output` and writes solution data to this directory.
- **-verbose**, displays extended information regarding the thermal simulation.
- **-output_precision** <**value**> sets the precision of the solution output. The default **value** = 6, i.e., output 6 digits of the solution.
- **-timestep_method** <**method**> sets the time step method of the simulation. The valid method names are **method**= be, bdf2, tr, stabtr, imr. Default **method**=be.
- **-dt** <**value**>, sets the time step size $\Delta t =$ **value**. If Δt is not specified, then it defaults to the atomic time step size.

- **-output_frequency** <**value**> sets the frequency of the solution output to a file. The default is **value** = 100, i.e., the solution is output every 100 time steps. A value of 1 would output the solution at every time step, a potentially expensive operation for large-scale thermal simulations with fine grids and a large number of time steps. It is advisable to tune this parameter based on the mesh size and total number of time steps.
- **-fdi** <**value**> sets the frequency of the finite difference interrupt (fdi) k^* in the stabilized trapezoid rule (Section 8.1.2). The default is **value** = 10, i.e., every 10 time steps perform the finite difference interrupt ([22]) when used in adaptive time stepping. The use of finite difference interrupts allows for the adaptive stabilized trapezoid rule to take larger time steps when the solution tends to a steady state. The use of this method requires additional storage of previous solutions and more frequent finite difference interrupts can increase the computational time of the solution. Note that if fdi = ∞, i.e., no finite difference interrupts, the standard trapezoid rule is recovered.
- **-solver_tolerance** <**value**> sets the CG solver tolerance to value. The default is **value** = 10^{-8}. A reasonable range of tolerance values is $[10^{-6}, 10^{-10}]$ but can be adjusted depending on the accuracy requirements of the problem.

8.2.3 Mode 2 flags

In addition to the flags described in Section 8.2.2, in Mode 2 an additional flag is available.

- **-eps_t** < **value** > sets the local truncation error tolerance to ε_t = **value**. Note that $0 <$ **value** < 1. The default is **value**=10^{-4}. The user-prescribed adaptive time stepping tolerance ε_t influences the size of the time step. Small tolerances, i.e., $\varepsilon_t \to 0$ provide more accurate thermal simulations but at the cost of having to use smaller time step sizes, which increases the execution time of thermal simulations. Larger tolerance values, i.e., $\varepsilon_t \approx \mathcal{O}(10^{-2})$ allow for larger, less accurate time steps. Generally, a reduction in ε_t by an order of magnitude results in an increase in the number of time steps by a factor $\log_{q+1}(10)$, where q is the asymptotic order of the time stepping method. A reasonable range of values for this tolerance is $[10^{-2}, 10^{-6}]$.

8.2.4 Mode 3 flags

In addition to the flags described in Section 8.2.2, in Mode 3 the following additional flags are available.

- **-amr_frequency** < **value** > sets the frequency of adaptive mesh refinements. The default is **value** = 3. Decreasing this value improves the accuracy of the solution but at an increased computational cost, resulting in longer solution times. Increasing this value results in fast simulation times while possibly sacrificing accuracy of the solution.
- **-amr_refine** <**value**> sets the AMR refinement percentage to value between [0,1]. A value of 1 corresponds to uniform refinement and 0 corresponds to no refinement. The default value is 0.6.
- **-amr_merge** <**value**> sets the AMR merging percentage to a value between [0,1]. A value of 1 corresponds to uniform merging and 0 corresponds to no merging. The default value is 0.4.

8.2.5 Mode 4 flags

Mode 4 is the fully adaptive spatiotemporal mode of operation and simultaneously combines the adaptive time stepping with adaptive mesh refinement described in Section 8.1.3. The optional flags to configure

thermal simulations in Mode 4 include the union of flags available in Modes 1, 2, and 3 presented in Sections 8.2.2, 8.2.3, and 8.2.4, respectively.

8.2.6 Model type flag

The model type flag determines whether a simulation used a linear model (Section 8.1.2), a nonlinear model (Section 8.1.4), or a linearized model (Section 8.1.5). The material properties in linear thermal simulations are constant and temperature-independent. For nonlinear thermal simulations, the material properties are temperature-dependent and the MTA assumes that this relationship is linear. This is a valid assumption for the materials involved and for the operating temperature range of modern ICs. To change the model type, set the following flag in the **$heat_configuration$** variable:

-model_type <type>,

where **type** is linear, nonlinear, or linearized. The default model type is linear.

8.2.7 Output flags

The default output of the MTA is a series of **solution.vtu** files which can be visualized with any external tool that supports this format. To control the solution output the following flags are available.

- **-output_frequency <value>** sets the frequency of the solution output to a file. The default is **value** = 100, i.e., the solution is output every 100 time steps.
- **-output_precision <value>** sets the precision of the solution output. The default is **value** = 6, i.e., output 6 digits of the solution.
- **-last_step** outputs the solution data at the final step of the simulation only.
- **-output_simple <output_rang.rng>** reads the output range file and then outputs the temperature values at the points that lie within the ranges specified in the .rng file. No .vtu file is generated with this flag. Enabling this option provides significant computational savings when the meshes contain a large number of nodes.

8.2.8 Checkpoint flags

Users may wish to start a new transient simulation where the initial temperature profile starts from a non constant value. This function allows the user to output the temperature of a thermal simulation and use these values as the initial condition for a new simulation. The following flags describe how to enable these features.

- **-dump_file <dump_file.dat>** dumps a file with the temperature profile at the last step of the simulation.
- **-resume_file <resume.dat>** reads in the initial temperature profile specified in the resume file.

9 Mesh generation module

The mesh generator of the MTA provides a conversion from the hierarchical XML file to a .msh file. The format of the .msh file is based on the open-source mesh generation tool GMSH [15]. This approach allows

users to easily visualize their meshes with the GMSH graphical user interface. The msh_gen executable located in the `bin/` directory parses the XML file and based on the circuit components defined in the hierarchy generates the finite element mesh. The mesh file contains both node and element information. The base length unit in the mesh file is nanometers, which means that the smallest structure that can be represented in the XML is 1 nm in size.

If a component has subdomain definitions, these override the domain definition. Thus, there is only one unique spatial definition of a component and dual definitions of subdomains are avoided. The generated meshes are aligned with the boundaries of the subdomains. The mesh generator also provides built in refinement capabilities. Refining the mesh using this function is strongly suggested if the initial mesh is too coarse. For meshes with overhanging components such as a heat spreader and/or heat sink, the mesh generator offers local refinements near these overhanging corners and edges. Enabling such refinements improves the accuracy of the solution near these features; see [12, Sec. 1.5] for a discussion on this phenomenon.

9.1 Mesh refinement flags

The user can pass the following mesh generation configuration flags to the **$mesh_configuration$** variable (cf. Section 6).

- **-mesh <output_file_name.msh>** defines the output file name of the mesh. The default file name is **defaultMeshName.msh**.
- **-level <int>** sets the granularity level in the XML hierarchy. The default is 0 which means parse the entire hierarchy.
- **-refine <int>** sets the number of pre-simulation uniform mesh refinements. The default value is 0.
- **-ufixmesh** disables mesh corrections near corners. For applications with very fine initial meshes, there is no need to execute this function.
- **-ufixstretch**, disables mesh corrections for a stretched mesh. For applications with very refined meshes, there is no need for this option.
- **-ucheckgrid** disables the check grid function which reports if the current mesh has a grid refined below 10 nm. This argument is used as a warning and advises that no further refinement of the grid because the mesh width is already too small. It also gives a hint to check the mesh again to avoid wrong structure definitions.
- **-CFMU <value>** adds mesh lines at a distance of **value** at the corners and overhanging boundaries of the mesh to improve accuracy. The result of enabling this flag on an example mesh is shown by the red and green dotted lines in Figure 14. The default value is 10,000 nm (or 10 μm) from the boundary.
- **-RCFMU <int>**, creates additional mesh lines near the boundary (based on the CFMU flag). The default value is 1, i.e., add 1 additional grid line in both the x- and y-directions of the grid. If the value is set to two, then it performs two refinements as shown by the red and green dotted lines in Figure 14.
- **-FMST <int>**, creates a fixed threshold to fix stretched meshes by adding extra refinements to stretched mesh elements larger than this threshold. This is illustrated by the blue dotted lines in Figure 14. The default threshold is 10^6 nm (or 1 mm).
- **-fixlevel <N>**, sets up the parsing level in the XML hierarchy for fixing corners. Any IC components with the hierarchy level larger than the set value **N** will result in local refinement near their corners.
- **-deltat** returns the maximum and minimum element sizes in the entire mesh.

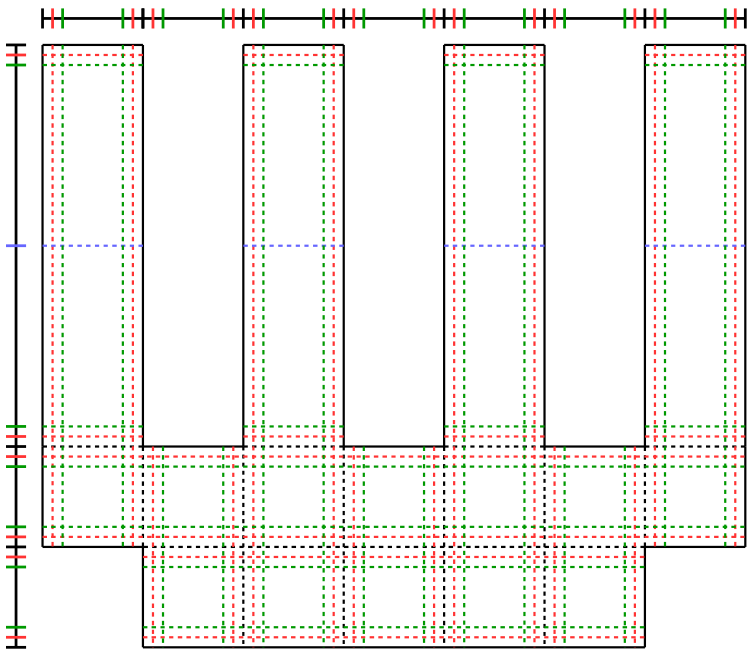


Figure 14: The mesh of refinements produced using the CFMU (red/green), RCFMU (red/green), and FMST (blue) flags.

9.2 Cell-level mesh reduction flags

Due to the nanoscale spatial features of the circuit, a cell-level mesh can contain tens (even hundreds) of millions of nodes, which can make thermal simulations computationally infeasible. The MTA offers several techniques to reduce the size of the computational mesh while still maintaining the accuracy of the circuits temperature profile. In this section we describe how to enable the various mesh reduction techniques. For full mathematical details the reader is referred to [8].

For cell-level thermal simulations, the mesh generator outputs an option (.opt) file that is used to calibrate the thermal simulation. To specify a name for this file the following flags can be passed to the **$msh_configuration$** variable in the run flow Perl script. The second flag sets the parsing level in the XML hierarchy for local refinements near corners.

- **-optoutput <output_file_name.opt>** Output the .opt file to the output directory for cell-level simulations with a specific name. The default name is "default.opt."
- **-optlevel <N>** Set the parsing level in the XML hierarchy for fixing corners. Any components below the set level are homogenized.

In [8] we proposed and implemented three algorithms for reducing the size of the computational mesh. They are based on weighting grid lines based on purely spatial information, weighting grid lines based on power density information, and a hybrid mode that combines the two approaches. How to enable these modes with flags are described in the following subsections.

9.2.1 Spatially weighted grid lines

To setup the spatially weighted grid lines scheme the user passes these flags to the mesh configuration variable.

- **-cellopt** enables the cell-level homogenization based on the weighted grid lines scheme.
- **-CELLTH <N>** sets the threshold for the weighted grid lines scheme. The algorithm counts for each grid line the number of cells it is incident to. Then all grid lines where this number is higher than the threshold are kept in the mesh. The default value is $\mathbf{N} = 10$.

9.2.2 Power density weighted grid lines

To set up the power density weighted grid lines mesh reduction scheme the user passes the following flags to the mesh configuration variable.

- **-poweropt <power.avg>** switches on cell-level homogenization based on the power density associated to each grid line in the average power file.
- **-POWERDENSITYTH <value>** sets the threshold value for the power density. Based on the power densities of the cells associated with each grid line an average (total) power density of a line is computed. The grid lines with associated power values above the threshold are kept in the mesh. The default value is 1, though this value is problem dependent.

9.2.3 Hybrid mode

To set up the hybrid mode, which is a combination of the previous two mesh reduction techniques, pass the following flags to the mesh configuration variable.

- **-smartopt <power.avg>** enables cell-level homogenization based on the rank of the grid lines. The rank is determined by both the weight of the grid lines and the power density of the grid line.
- **-SMARTRATE <value>** preserves the percent of grid lines specified by the value. The value is a number between [0,1]. The default value is 0.5.

10 External input files

In this section we briefly cover the external files that are supplied as an input to the MTA. They are the powertrace file which contains all of the power information of the active components of the circuit, and the industrial standard DEF and LEF files which describe the geometry and layout of the standard cells when a cell-level simulation is executed.

10.1 Powertrace

The powertrace (.ptrace) file is a list of the power values (in units W) that are dissipated to each active component of the circuit. The base unit of time for which power values are supplied to the circuit is the *atomic time step size* Δt_a. The powertrace file consists of column vectors, where different columns correspond to different active components of the circuit, and each column entry is the corresponding power value at a specific time. The powertrace files are generated by architecture simulators coupled with powertrace converters, or power estimators for ICs, such as PrimetimePX [29] and, therefore, follow the same structure.

10.2 DEF and LEF

For cell-level simulations, the MTA takes as input the industrial standard design exchange format (DEF) and library exchange format (LEF) files and converts them into a die XML file that can be used in cell-level thermal simulations.

References

[1] U. Ayachit. *The ParaView Guide: A Parallel Visualization Application*. Kitware, Inc., 2015.

[2] S. Balay, S. Abhyankar, M. F. Adams, J. Brown, P. Brune, K. Buschelman, L. Dalcin, V. Eijkhout, W. D. Gropp, D. Kaushik, M. G. Knepley, L. C. McInnes, K. Rupp, B. F. Smith, S. Zampini, H. Zhang, and H. Zhang. PETSc Web page. http://www.mcs.anl.gov/petsc, 2016.

[3] S. Balay, W. D. Gropp, L. C. McInnes, and B. F. Smith. Efficient management of parallelism in object oriented numerical software libraries. In E. Arge, A. M. Bruaset, and H. P. Langtangen, editors, *Modern Software Tools in Scientific Computing*, pages 163–202. Birkhäuser Press, 1997.

[4] W. Bangerth, R. Hartmann, and G. Kanschat. deal.II – a General Purpose Object Oriented Finite Element Library. *ACM Transactions on Mathematical Software*, 33(4):24/1–24/27, August 2007.

[5] W. Bangerth, T. Heister, and G. Kanschat. deal.II Differential Equations Analysis Library, Technical Reference. http://www.dealii.org.

[6] J. Boyle, M. Mihajlović, and J. Scott. HSL_MI20: An Efficient AMG Preconditioner for Finite Element Problems in 3D. *International Journal for Numerical Methods in Engineering*, 82(1):64–98, October 2010.

[7] W. L. Briggs, V. E. Henson, and S. F. McCormick. *A Multigrid Tutorial*. SIAM Philadelphia, 2nd edition, 2000.

[8] Y.-C. Chen, S. Ladenheim, M. Mihajlović, and V. F. Pavlidis. Computationally Efficient Standard-Cell FEM-based Thermal Analysis. In *Proceedings of the International Conference on Computer-Aided Design*, 2017.

[9] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. North-Holland, Amsterdam, 1978.

[10] W. Dörfler. A Convergent Adaptive Algorithm for Poisson's Equation. *SIAM Journal on Numerical Analysis*, 33:1106–1124, June 1996.

[11] D. Edwards. Semiconductor and IC Package Thermal Metrics, 2012.

[12] H. C. Elman, D. J. Silvester, and A. J. Wathen. *Finite Elements and Fast Iterative Solvers: with applications in incompressible fluid dynamics*. Oxford University Press, 2nd edition, 2014.

[13] L. C. Evans. *Partial Differential Equations*, volume 19 of *Graduate studies in mathematics*. American Mathematical Society, Providence, RI, 1998.

[14] S. J. Farlow. *Partial differential equations for scientists and engineers*. Dover Publications, Inc., New York, 2012.

[15] C. Geuzaine and J.-F. Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009.

[16] M. S. Gockenbach. *Understanding and Implementing the Finite Element Method*. SIAM Philadelphia, 2006.

[17] P. Gresho and R. Sani. *Incompressible Flow and the Finite Element Method: Advection-Diffusion and Isothermal Laminar Flow*. John Wiley & Sons, Inc., New York, 1998.

[18] M. R. Hestenes and E. Stiefel. Methods of Conjugate Gradients for Solving Linear Systems. *Journal of Research of the National Bureau of Standards*, 49:409–436, December 1952.

[19] W. Huang, S. Ghosh, S. Velusamy, K. Sankaranarayanan, K. Skadron, and M. R. Stan. Hotspot 6.0. http://lava.cs.virginia.edu/HotSpot/, 2016.

[20] D. W. Kelly, J. P. De S. R. Gago, O. C. Zienkiewicz, and I. Babuška. A Posteriori Error Analysis and Adaptive Processes in the Finite Element Method: Part I–Error Analysis. *International Journal for Numerical Methods in Engineering*, 19:1593–1619, November 1983.

[21] S. Ladenheim, Y.-C. Chen, M. Mihajlović, and V. F. Pavlidis. IC Thermal Analyzer for Versatile 3-D Structures Using Multigrid Preconditioned Krylov Methods. In *Proceedings of the International Conference on Computer-Aided Design*, pages 123:1–123:8, November 2016.

[22] J. A. Lee, J. Nam, and M. Pasquali. A New Stabilization of Adaptive Step Trapezoid Rule Based on Finite Difference Interrupts. *SIAM Journal on Scientific Computing*, 37(2):A725–A746, March 2015.

[23] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-state and Time-dependent Problems*, volume 98. SIAM Philadelphia, 2007.

[24] X. S. Li, J. W. Demmel, J. R. Gilbert, L. Grigori, M. Shao, and I. Yamazaki. SuperLU Users' Guide, 2011. http://crd.lbl.gov/~xiaoye/SuperLU/.

[25] J. Meng, K. Kawakami, and A. K. Coskun. Optimizing Energy Efficiency of 3-D Multicore Systems with Stacked DRAM under Power and Thermal Constraints. In *Proceedings of the 49th Annual Design Automation Conference*, pages 648–655. ACM, 2012.

[26] Y. S. Touloukian and C. Ho. *Thermophysical Properties of Matter: Specific Heat - Metallic Elements and Alloys*, volume 4. IFI/Plenum, New York, 1970.

[27] Y. S. Touloukian, R. Powell, C. Ho, and P. Klemens. *Thermophysical Properties of Matter: Thermal Conductivity - Metallic Elements and Alloys*, volume 1. IFI/Plenum, New York, 1970.

[28] U. M. Yang et al. BoomerAMG: A Parallel Algebraic Multigrid Solver and Preconditioner. *Applied Numerical Mathematics*, 41(1):155–177, 2002.

[29] G. Yip. Expanding the Synopsys Primetime Solution with Power Analysis. http://www.synopsys.com, 2006.